

Data Structure hw2

杨其霄202300091132信计

Q3

(2)

(2) 在顺序表中删除所有元素值为 x 的元素, 要求空间复杂度为 $O(1)$ 。

补: 平均时间复杂度 $O(n)$

基本思路

关键在于将删除元素之后的元素进行前移, “删除”完之后再逐个前移会导致 $O(n^2)$ 的复杂度, 因此我们考虑交换元素而非单独的搬移。

注意到这道题可以用双指针解决, 我们基本的思路就是维护两个指针 `right` 和 `left`, 左指针指向处理好的序列尾部, 右指针指向待处理序列的头部, 均初始化为0。右指针不断向右移动, 每次遇到非 x , 就将左右指针对应的数交换, 同时左指针右移。由于左指针左边均非 x , 右指针左边直到左指针都是 x , 因此每次交换都是将左指针的 x 和右指针的非 x 交换。

题目要求 $O(1)$ 空间复杂度则不考虑缩容操作, 交换完成之后直接将`length`减去 x 的数量即可, 末尾的一堆 x 不会影响顺序表之后的基本操作。

复杂度: 算法只遍历了一遍数组, `swap`操作是 $O(1)$ 的, 总的复杂度就是 $O(n)$

伪代码

```
1 void swap(T& x, T& y) {
2     T tmp = x;
3     x = y;
4     y = tmp;
5 }
6
7 void removex(T x) {
8     // data and length are attributes of SeqList
9     int left = 0;
10    int right = 0;
11    int count = 0;
12    while (right < length) {
13        if (data[right] != x) {
14            swap(data[left], data[right]);
15            left++;
16            count++;
17        }
18        right++;
19    }
20    length = count;
21 }
```

代码

关键部分同上，这里给出test代码。

```
1  #include "../Lists/SeqList.h"
2
3  int main() {
4      int a[5] = {1, 2, 3, 4, 5};
5      int b[10] = {0, 1, 0, 1, 1, 0, 0, 1, 0, 1};
6      SeqList<int> s1(a, 5);
7      SeqList<int> s2(b, 10);
8      s1.print();
9      s1.removeX(3); // remove 3 from s1
10     s1.print();
11     s1.removeX(6); // remove 6 from s1
12     s1.print();
13     s2.print();
14     s2.removeX(0); // remove 0 from s2
15     s2.print();
16     s2.removeX(1); // remove 1 from s2
17     s2.print();
18     return 0;
19 }
```

输出：

```
PS D:\CS\Data-structure-sp26\hw2> .\a.exe
```

```
1 2 3 4 5
```

```
1 2 4 5
```

```
1 2 4 5
```

```
0 1 0 1 1 0 0 1 0 1
```

```
1 1 1 1 1
```

(3)

(3) 试分别以顺序表和单链表作存储结构，各编写一个实现线性表就地逆置的算法。

基本思路

对于顺序表，直接维护两个指针 `left` 和 `right`，当 `left < right` 的时候交换元素即可。对于链表，遍历所有节点，将当前节点的 `next` 置为上一节点即可。由于是单链表，需要额外维护一个 `prev` 指针。

复杂度：都只需要遍历一遍容器，都是 $O(n)$

伪代码

```
1  /**
2   * reverse of SeqList
3   */
4  void reverse() {
5      int left = 0;
6      int right = length - 1;
7      while (left < right) {
8          swap(data[left], data[right]);
9          left++;
10         right--;
11     }
12 }
```

```
1  /**
2   * reverse of SLList
3   * could use recursive instead.
4   */
5  void reverse() {
6      Node* prev = nullptr;
7      Node* curr = sentinel->next;
8      while (curr != nullptr) {
9          Node* next = curr->next; // tmp node
10         curr->next = prev; // change curr's next to prev
11         prev = curr; // now curr is the 'old' node
12         curr = next; // curr = curr->next
13     }
14     sentinel->next = prev;
15 }
```

代码

关键代码如上，test代码：

```
1  #include "../Lists/SLList.h"
2  #include "../Lists/SeqList.h"
3
4  int main() {
5      int a[5]{1, 2, 3, 4, 5};
6      SeqList<int> s(a, 5);
7      s.reverse();
8      s.print();
9
10     SLList<int> sl_list(a, 5);
11     sl_list.reverse();
12     sl_list.print();
13     return 0;
14 }
```

结果：

```
PS D:\CS\Data-structure-sp26\hw2> g++ .\q3-3.cpp -o q3-3.exe
```

```
PS D:\CS\Data-structure-sp26\hw2> .\q3-3.exe
```

```
5 4 3 2 1
```

```
5->4->3->2->1
```

(5)

(5) 给定一个带头结点的单链表,设计算法按递增次序输出单链表中各结点的数据元素,并释放结点所占的存储空间(要求:不允许使用数组作辅助空间)。

基本思路

维护两个指针 `prev` 和 `curr`, 分别指向最小节点的前置节点和当前节点。遍历链表, 当 `curr->next` 值小于 `prev->next` 值时, 更新 `prev = curr`。最后用 `minNode = prev->next` 暂存最小节点, 前置节点链接到 `minNode` 的下一节点, 输出值并删除 `minNode` 即可。

复杂度: 依次遍历至多 $n, n-1, \dots, 1$ 个节点, 复杂度 $O(n^2)$

伪代码

```
1 while (sentinel->next != nullptr) {
2     // when SLList is not empty
3     Node* prev = sentinel; // prev node of minNode
4     Node* curr = sentinel;
5     while (curr->next != nullptr) {
6         if (curr->next->data < prev->next->data) {
7             prev = curr;
8         }
9         curr = curr->next;
10    }
11    Node* minNode = prev->next; // minNode is the next node of prev
12    prev->next = prev->next->next;
13    std::cout << minNode->data << " ";
14    delete minNode;
15    size_--;
16 }
```

代码

关键代码如上, test代码:

```
1 #include "../Lists/SLList.h"
2
3 int main() {
4     int a[10] = {3, 4, 4, 2, 1, 7, 8, 20, 15, 0};
5     SLList<int> s(a, 10);
6     s.print();
7     s.increase();
8     std::cout << "isEmpty: " << s.empty();
9     return 0;
10 }
```

结果:

```
PS D:\CS\Data-structure-sp26\hw2> g++ .\q3-5.cpp -o q3-5.exe
```

```
PS D:\CS\Data-structure-sp26\hw2> ./q3-5.exe
```

```
3->4->4->2->1->7->8->20->15->0
```

```
0 1 2 3 4 4 7 8 15 20
```

```
isEmpty: 1
```

(6)

(6) 设单链表以非递减有序排列,设计算法实现在单链表中删去值相同的多余结点。

基本思路

和3-2的思路基本一致,利用双指针 `left` 和 `right` 滑动窗口,左指针是已经两两互异的节点的末节点,右指针左边直到左指针值都相同。

当左右指针 `data` 不同时, `left->next = right`, 并更新 `left = right`;

相同时,先用临时指针暂存右指针,右指针更新为下一元素后删除临时指针;

最后断尾,将左指针下一节点设为 `nullptr`。

复杂度: 只用遍历一遍链表,复杂度 $O(n)$

伪代码

```
1 void delRedunant() {
2     if (sentinel->next == nullptr) {
3         return;
4     }
5     Node* left = sentinel->next;
6     Node* right = sentinel->next->next;
7     while (right != nullptr) {
8         if (right->data != left->data) {
9             left->next = right;
10            left = right;
11            right = right->next;
12            continue;
13        }
14        Node* tmp = right;
15        right = right->next;
16        delete tmp;
17        size--;
18    }
19    left->next = nullptr;
20 }
```

代码

test代码:

```
1  #include "../Lists/SLList.h"
2
3  int main() {
4      int a[13] = {1, 1, 2, 2, 3, 3, 3, 4, 4, 5, 5, 5, 6};
5      SLList<int> s(a, 13);
6      s.print();
7      s.delRedundant();
8      s.print();
9      return 0;
10 }
```

输出:

```
PS D:\CS\Data-structure-sp26\hw2> g++ .\q3-6.cpp -o q3-6.exe
```

```
PS D:\CS\Data-structure-sp26\hw2> ./q3-6.exe
```

```
1->1->2->2->3->3->3->4->4->5->5->5->6
```

```
1->2->3->4->5->6
```

补充

关键代码作为 SLList 和 SeqList 的成员函数实现, 详见 ".\Lists\SLList.cpp" 和 ".\Lists\SeqList.cpp"